Will Young
Student ID: 924230057
Week 12 Report
4/17/2026

Hello Professor,

Below is an update for my week 12 progress. In addition, I have pushed my updates to my github repo: https://github.com/Woyoung21/LastMile-Sec. For week 12, the main tasks were to design the review interface where a human engineer validates the proposed remediation and run a full end-to-end test with real vulnerability reports. This process turned out to be a bit more difficult than I imagined for a few different reasons which I will discuss. The frontend did finally get built, and I'm very happy with the design. I will discuss some of the choices I made for the front end and include a quick demo with screen shots in this document.

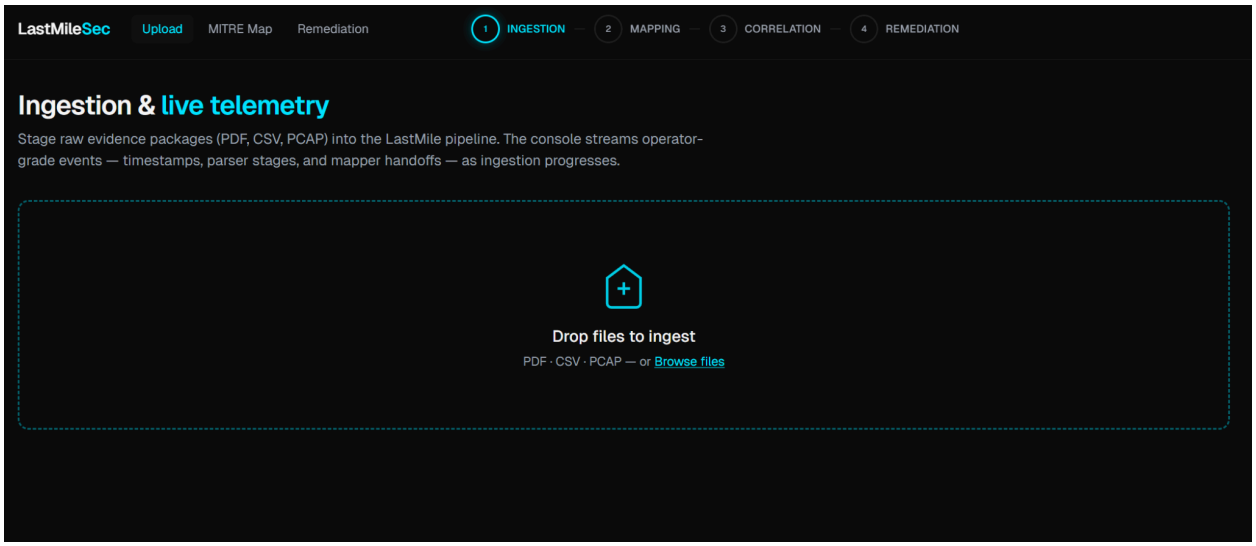
One large issue I experienced was a service degradation for Gemini's API that has been active all week, causing the portions of my project that relied on Gemini to fail. This was well documented in many of Google's developer forums and even their github, with no response from Google's staff. Because of this, I had to spend a substantial amount of time to create robust backoff/retry loops in my backend code so that those portions would eventually execute without failing because of all the 503 errors. This was the primary solution I saw in these forums. Another issue was my early design choices affected my development of the frontend. The way I developed the backend was very segmented, each section was it's own piece, that fed the next section some new metadata fields in the shared JSON. What would've been a better option would've been to develop this as a comprehensive project, containerized to keep all the backend together. This would've shielded the backend's library versions from anything I would later create in the frontend, as that would also be containerized in its own environment.

Initially, when I was building out the front end, the dependencies that were installed for some of the front end components broke my backend. Actian needs its own library version, Neo4j does, CUDA does for my local MISTRAL model, langextract needs its own version of its dependencies.The backend components are incredibly delicate and need very specific versions of specific libraries to function. I learned this the hard way and actually had to revert my code to what was in git and pivot my frontend development. I forgot that these components were so delicate, mainly because I hadn't worked in some of these sections for a few weeks. Additionally, after I reverted my code, I had to reinstall all of the specific dependencies in a specific order, and then ensure each section ran like normal. During this process I hit a lot of 503 errors as I mentioned earlier, which caused its own problems.

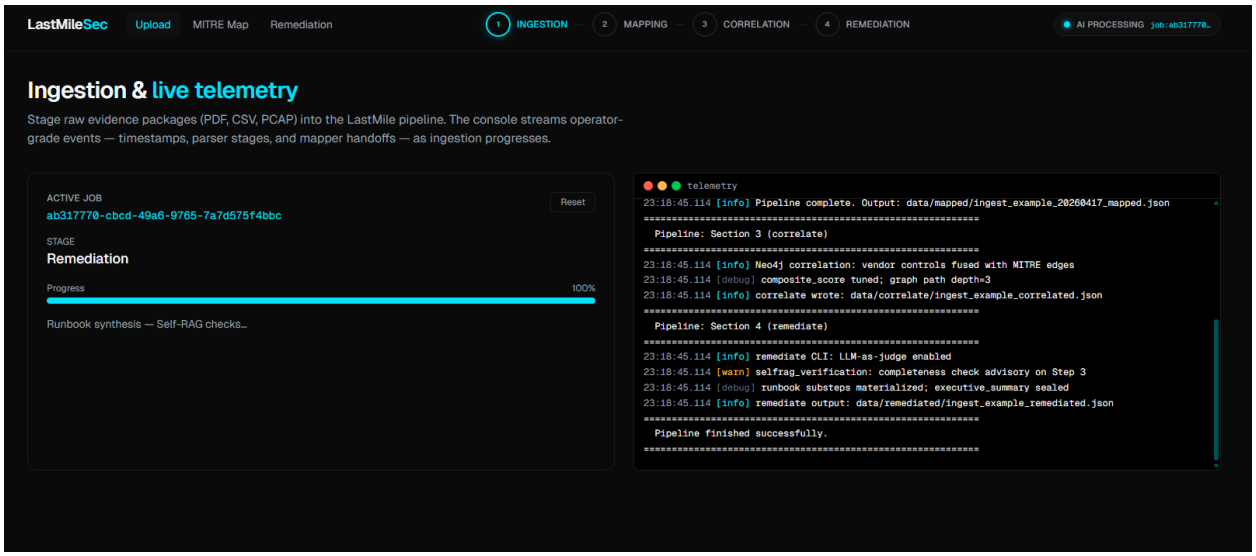
Ultimately the frontend was finally built, and I'm very happy with the outcome. Based on my initial experience breaking my backend, I made a big change. After I reverted back to a clean project, I built the entire frontend in a docker container. Everything frontend was compartmentalized from the backend, this way I wouldn't run into the same issues I had the first time around. For the design, I needed a dynamic setup that would visually tell the story of the backend. My design choices for the front end changed quite a bit because I am moving onto the next round for the SFSU AI Awards. For the next round I need to create a 5 minute demo video, so I needed a few pages that were very dynamic.

Will Young
Student ID: 924230057
Week 12 Report
4/17/2026

I started off with an upload page to start the work flow. This is where the engineer would submit the vulnerability report or PCAP file. To start the process, you upload a file.

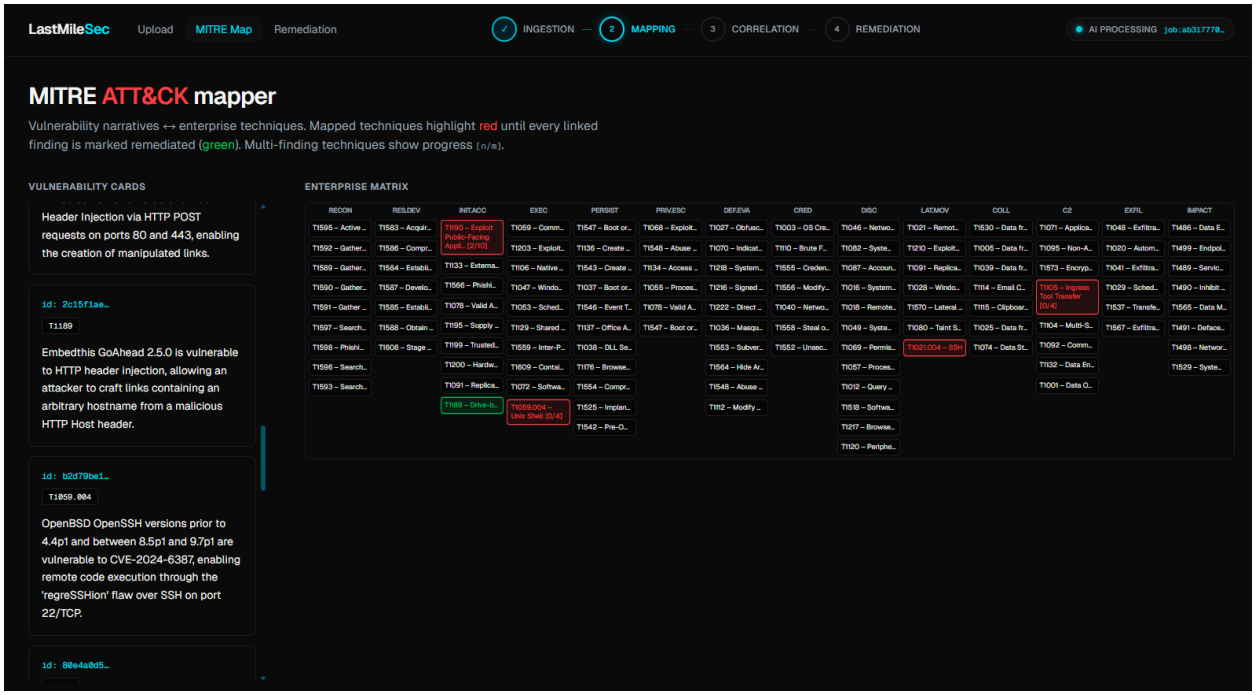


Once that happens, the screen will start to show you a progress bar and a telemetry terminal which would mimic the output we would see in our IDE terminal. At the moment, this is simulated. These backend runs, depending on how many events, can take over an hour from start to finish. Because of this, I decided to mock the telemetry output from past runs.

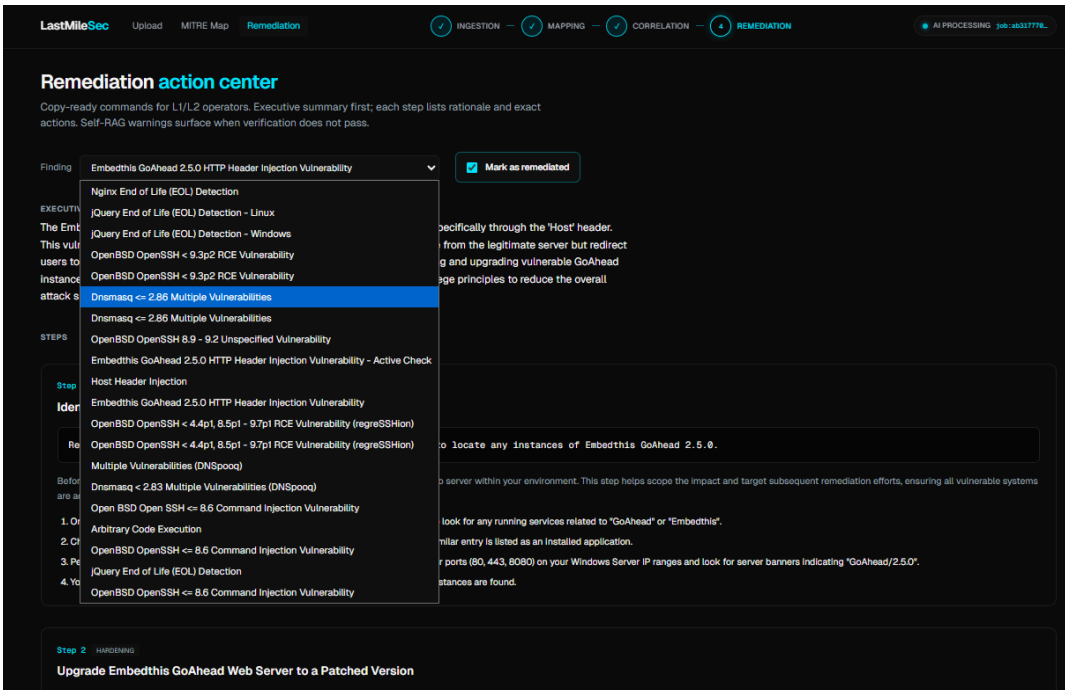


On the right is the telemetry terminal, and the text gradually gets printed to the screen, simulating backend activity moving from one section to the next. Additionally, there are 4 sections up top, and as we progress through the backend stages, they lite up as another visual aid to let you know where we are at. The second page is a MITRE Map, one of the most visual pages.

Will Young
Student ID: 924230057
Week 12 Report
4/17/2026

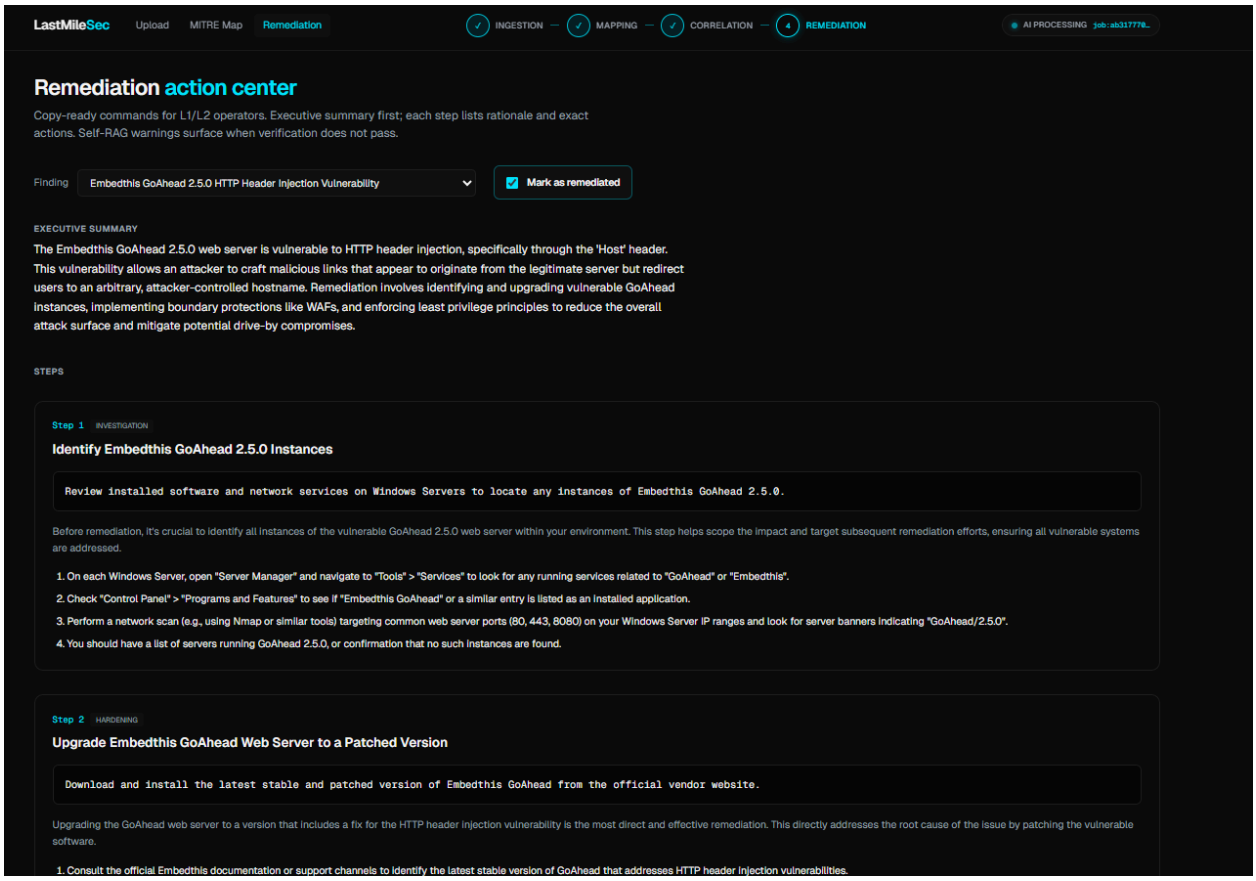


This is where I wanted a dynamic visual, and it is designed after MITRE's own TTP wall (https://attack.mitre.org/). On the left are the vulnerability cards, which is real data from the run that was completed. The sentence in each card is the Reporter Agents technical summary and the ID's are from the Reporter. This map is dynamic, where the MITRE IDs that were identified from the scan show up as red on this page. On the following page, we can click a "remediated" check box, and if all the proposed fixes are remediated for the MITRE ID, the box turns green.



Will Young
Student ID: 924230057
Week 12 Report
4/17/2026

The final page is the remediation center. Again, this is real data from the latest scan in our /data/remediated folder. The drop down shows all of the events, and you can select one to see an overview and the steps to remediate the problem.



At the top, we get a nice executive summary overview of the vulnerability. This vulnerability that was identified has about 6 steps to fix, each with detailed substeps to help the L1/L2 engineer to complete. You can also see the “Mark as Remediated” checkbox I was referencing earlier.

I will continue to refine this frontend as we move forward, but overall I am very pleased with how it looks at the moment. Ideally the upload page will trigger the backend to start a new run and the terminal output will be exactly what is being displayed during a real end-to-end run. Looking forward to next week, I will begin drafting my report for this project. As this timeline starts to wind down, I will also focus on overall improvements in accuracy and stability.